Vansh Thakral

500125288

Batch-1

Lab Exercise 15- Creating ConfigMaps in Kubernetes

In this lab exercise, you will learn how to create ConfigMaps in Kubernetes using different methods: from literal values, from a file, from an environment file, and from multiple files. ConfigMaps allow you to decouple configuration artifacts from image content to keep containerized applications portable.

Step 1: Set Up Kubernetes Cluster

Ensure you have access to a Kubernetes cluster. You can use a local setup with Minikube, kind, or use a cloud-based Kubernetes service.

Step 2: Create a ConfigMap from Literal Values

Create a ConfigMap named example-configmap with literal values:

```
kubectl create configmap example-configmap --from-literal=key1=value1 --from-literal=key2=value2
```

Verify the ConfigMap:

```
kubectl get configmap example-configmap -o yaml
```

You should see the example-configmap with the keys and values specified.

```
configmap/example-configmap created
(base) vanshthakral@Vanshs-MacBook-Air Devsecops15 % kubectl get configmap example-configmap
kubectl describe configmap example-configmap
kubectl get configmap example-configmap -o yaml
NAME                DATA   AGE
example-configmap   2      10s
Name:         example-configmap
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
key1:
----
value1

key2:
----
value2


BinaryData
====

Events:  <none>
apiVersion: v1
data:
  key1: value1
  key2: value2
kind: ConfigMap
metadata:
  creationTimestamp: "2026-04-13T05:00:52Z"
  name: example-configmap
  namespace: default
  resourceVersion: "20080"
  uid: 4ab4ade9-f8dc-4a00-89d7-1b342c4ec25e
```

Step 3: Create a ConfigMap from a File

Create a file named **config.txt** with the following content:

```
key1=value1
key2=value2
```

Create a ConfigMap named file-configmap from the file:

```
kubectl create configmap file-configmap --from-file=config.txt
```

Verify the ConfigMap:

```
kubectl get configmap file-configmap -o yaml
```

You should see the file-configmap with the content of config.txt.

```
(base) vanshthakral@Vanshs-MacBook-Air Devsecops15 % kubectl get configmap file-configmap -o yaml
apiVersion: v1
data:
  config.txt: |
    key1=value1
    key2=value2
kind: ConfigMap
metadata:
  creationTimestamp: "2026-04-13T05:02:12Z"
  name: file-configmap
  namespace: default
  resourceVersion: "20151"
  uid: ffefdc5e-a35f-4520-affa-53275af89813
```

Step 4: Create a ConfigMap from an Environment File

Create an environment file named **env-config.env** with the following content:

```
ENV_VAR1=value1
ENV_VAR2=value2
```

Create a ConfigMap named env-configmap from the environment file:

```
kubectl create configmap env-configmap --from-env-file=env-config.env
```

Verify the ConfigMap:

```
kubectl get configmap env-configmap -o yaml
```

You should see the env-configmap with the environment variables specified in file.

```
(base) vanshthakral@Vanshs-MacBook-Air Devsecops15 % kubectl get configmap env-configmap -o yaml
apiVersion: v1
data:
  ENV_VAR1: value1
  ENV_VAR2: value2
kind: ConfigMap
metadata:
  creationTimestamp: "2026-04-13T05:03:22Z"
  name: env-configmap
  namespace: default
  resourceVersion: "20216"
  uid: 37b0f13d-3ed7-420c-8150-7258c13e3668
```

Step 5: Create a ConfigMap from Multiple Files

Create multiple files:

config1.txt with content:

```
key1=value1
```

config2.txt with content:

```
key2=value2
```

Create a ConfigMap named multi-file-configmap from multiple files:

```
kubectl create configmap multi-file-configmap --from-file=config1.txt --from-file=config2.txt
```

Verify the ConfigMap:

```
kubectl get configmap multi-file-configmap -o yaml
```

You should see the multi-file-configmap with the content of both config1.txt and config2.txt.

```
● (base) vanshthakral@Vanshs-MacBook-Air Devsecops15 % kubectl get configmap multi-file-configmap -o yaml
  apiVersion: v1
  data:
    config1.txt: |
      key1=value1
    config2.txt: |
      key2=value2
  kind: ConfigMap
  metadata:
    creationTimestamp: "2026-04-13T05:04:20Z"
    name: multi-file-configmap
    namespace: default
    resourceVersion: "20261"
    uid: 00fe96cf-2a57-4f91-bc97-ce75046bec09
```

Step 6: Use ConfigMaps in a Pod

Create a file named **pod-configmap.yaml** with the following content:

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-pod
spec:
  containers:
  - name: nginx
    image: nginx
    envFrom:
    - configMapRef:
        name: env-configmap
```

In this manifest:

- The Pod reads environment variables from the env-configmap.
- The Pod mounts the file-configmap at /etc/config.

Apply the manifest to create the Pod:

```
kubectl apply -f pod-configmap.yaml
```

Check the status of the Pod:

```
kubectl get pods
```

View the Pod logs to see the environment variables and file content:

```
kubectl exec -it configmap-pod -- printenv
```

You should see the environment variables and the content of the file-configmap.

```
● (base) vanshthakral@Vanshs-MacBook-Air Devsecops15 % kubectl exec -it configmap-pod -- printenv
  PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
  HOSTNAME=configmap-pod
  TERM=xterm
  ENV_VAR1=value1
  ENV_VAR2=value2
  KUBERNETES_SERVICE_HOST=10.96.0.1
  KUBERNETES_SERVICE_PORT=443
  KUBERNETES_SERVICE_PORT_HTTPS=443
  KUBERNETES_PORT=tcp://10.96.0.1:443
  KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
  KUBERNETES_PORT_443_TCP_PROTO=tcp
  KUBERNETES_PORT_443_TCP_PORT=443
  KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
  NGINX_VERSION=1.29.8
  NJS_VERSION=0.9.6
  NJS_RELEASE=1~trixie
  ACME_VERSION=0.3.1
  PKG_RELEASE=1~trixie
  DYNPKG_RELEASE=1~trixie
  HOME=/root
```

Step 7: Clean Up

After completing the exercise, clean up the resources created:

```
kubectl delete pod configmap-pod
kubectl delete configmap example-configmap
kubectl delete configmap file-configmap
kubectl delete configmap env-configmap
kubectl delete configmap multi-file-configmap
rm config.txt env-config.env config1.txt config2.txt
```

```
● (base) vanshthakral@Vanshs-MacBook-Air Devsecops15 % kubectl delete pod configmap-pod

  kubectl delete configmap example-configmap
  kubectl delete configmap file-configmap
  kubectl delete configmap env-configmap
  kubectl delete configmap multi-file-configmap

  rm config.txt env-config.env config1.txt config2.txt
  pod "configmap-pod" deleted from default namespace
  configmap "example-configmap" deleted from default namespace
  configmap "file-configmap" deleted from default namespace
  configmap "env-configmap" deleted from default namespace
  configmap "multi-file-configmap" deleted from default namespace
```